

# **ANEXOS DE TOW APP SYSTEMS**

**Autores: Gabriel Méndez, Paula Cardenas y Rubén  
Llanes**

**Tutor: Iván Noé Arévalo Vela**

**CFGS Desarrollo de Aplicaciones Multiplataforma  
Curso 2025-2026**

**IFP C/ Julián Camarillo  
Convocatoria de Presentación: Mayo 2026**



**TOW APP**

## ANEXOS

### Anexo 1: Conexión de la aplicación de Android y la aplicación web con Supabase

#### Aplicación Android

```
package ifp.tfg_grua_app

import ...

7 Usages
object SupabaseClient {
    7 Usages
    val client = createSupabaseClient(
        supabaseUrl = "https://sgyidsdrwqkqtqimozvm.supabase.co",
        supabaseKey = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJ...
    ) {
        install(plugin = Auth)
        install(plugin = Postgrest)
        install(plugin = Realtime)
    }
}
```

(SupabaseClient.kt) Este código configura la conexión entre la aplicación Android y Supabase, instalando los módulos necesarios para la autenticación, consultas a la base de datos y actualización en tiempo real.

#### Aplicación web

```
export class SupabaseService {
    private supabase: SupabaseClient;
    constructor() {
        this.supabase = createClient(
            'https://sgyidsdrwqkqtqimozvm.supabase.co',
            'sb_publishable_SOBQAb6mNQod2cCeG8QS-w_6XT1D-YJ'
            //publishable key
        );
    }
}
```

(supabase.service.ts) Este archivo gestiona todas las consultas y conexiones a la base de datos para todos los componentes de la aplicación web. Al ser inyectado en cualquier componente, le da acceso a los métodos CRUD.

```
export class AuthService {
    constructor(private supabaseService: SupabaseService) { }
}
```

## Anexo 2: Funciones de login del operador en Android

```
private fun validarLogin() {
    val correo = binding.email.text.toString().trim()
    val clave = binding.Password.text.toString().trim()

    // Validaciones simples
    if (correo.isEmpty()) {
        binding.email.error = "Introduce tu correo"; binding.email.requestFocus(); return
    }
    if (!android.util.Patterns.EMAIL_ADDRESS.matcher(input = correo).matches()) {
        binding.email.error = "Correo no valido"; binding.email.requestFocus(); return
    }
    if (clave.isEmpty()) {
        binding.Password.error = "Introduce tu contraseña"; binding.Password.requestFocus(); return
    }
    if (clave.length < 6) {
        binding.Password.error = "La contraseña debe tener al menos 6 caracteres"
        binding.Password.requestFocus(); return
    }

    // Busca en la tabla "usuarios" un registro con ese email + password
    loginJob = lifecycleScope.launch {
        try {
            val resultado = SupabaseClient.client
                .from(table = "usuarios")
                .select {
                    filter {
                        eq(column = "email", value = correo)
                        eq(column = "password", value = clave)
                    }
                }
        }
    }
}
```

([Login.kt](#)) El inicio de sesión valida los datos introducidos por el usuario y consulta la tabla usuarios en Supabase para comprobar si las credenciales son correctas.

```
data class Recogida(
    val id: Int? = null,
    @SerializedName(value = "nombre_cliente") val cliente: String? = null,
    @SerializedName(value = "vehiculo_matricula") val matricula: String? = null,
    @SerializedName(value = "observaciones") val motivo: String? = null,
    @SerializedName(value = "tel_cliente") val telefono: String? = null,
    @SerializedName(value = "ubicacion_recogida_lat") val lat: Double = 0.0,
    @SerializedName(value = "ubicacion_recogida_lng") val lng: Double = 0.0,
    @SerializedName(value = "ubicacion_destino_lat") val destinoLat: Double = 0.0,
    @SerializedName(value = "ubicacion_destino_lng") val destinoLng: Double = 0.0,
    @SerializedName(value = "num_empleado") val numEmpleado: Int? = null,
    @SerializedName(value = "vehiculo_recogido") val vehiculoRecogido: Boolean = false,
    val urgente: Boolean = false,
    val estado: String? = null
) {
    companion object {
        fun serializer(): KSerializer<Recogida>
    }
}
```

([Recogida.kt](#)) Esta clase representa el servicio de grúa obtenido desde la tabla de servicios Supabase. Los SerialName permiten relacionar los campos de las base de datos con los nombres usados en Kotlin.

```

private fun cargarRecogidas() {
    lifecycleScope.launch {
        try {
            val numEmpleado = SesionUsuario.getNumEmpleado( context = this@MainActivity)
            if (numEmpleado == null) {
                Toast.makeText( context = this@MainActivity,
                    text = "Sesión sin num_empleado, vuelve a iniciar sesión",
                    duration = Toast.LENGTH_LONG).show()
                return@launch
            }

            val recogidas = SupabaseClient.client
                .from( table = "servicios")
                .select { filter { eq( column = "num_empleado", value = numEmpleado) } }
                .decodeList<Recogida>()
                .filter { it.estado == "Sin empezar" || it.estado == "En curso" }
                .sortedByDescending { it.urgente }

            RecogidasRepo.Recogidas = recogidas

            binding.listaViajes.removeAllViews()
            for (viaje in recogidas) añadirTarjeta(viaje)

        } catch (e: Exception) {
            android.util.Log.e( tag = "MAIN", msg = "Error cargando recogidas", tr = e)
            Toast.makeText( context = this@MainActivity,
                text = "Error cargando recogidas: ${e.message}?: e.javaClass.simpleName}",
                duration = Toast.LENGTH_LONG).show()
        }
    }
}

```

([cargarRecogidas.kt](#)) Este código desde Supabase obtiene los servicios asignados al operador que ha iniciado sesión y los muestra en la pantalla de servicios de la aplicación.

```

// Cálculo de la ruta
// Llama a la API de Directions, parsea el JSON y actualiza la pantalla
2 Usages
private fun calcularRuta() {
    val origen = posicionActual ?: return
    binding.tvInstruccion.text = "Calculando ruta..."

    lifecycleScope.launch( context = Dispatchers.IO) {
        try {
            val url = "https://maps.googleapis.com/maps/api/directions/json" +
                "?origin=${origen.latitude},${origen.longitude}" +
                "&destination=${destino.latitude},${destino.longitude}" +
                "&mode=driving&language=es&key=$API_KEY"

            val json = OkHttpClient()
                .newCall(Request.Builder().url(url).build())
                .execute().body?.string() ?: return@launch

            val obj = JSONObject(json)
            if (obj.getString( name = "status") != "OK") {
                withContext( context = Dispatchers.Main) {
                    binding.tvInstruccion.text = "Error: ${obj.getString( name = "status")}"
                }
                return@launch
            }

            // Datos del primer "leg" de la primera ruta
            val tramo = obj.getJSONArray( name = "routes").getJSONObject( index = 0)
                .getJSONArray( name = "legs").getJSONObject( index = 0)
            val distancia = tramo.getJSONObject( name = "distance").getString( name = "text")
            val distanciaMetros = tramo.getJSONObject( name = "distance").getInt( name = "value")
            val tiempo = tramo.getJSONObject( name = "duration").getString( name = "text")
            val segundos = tramo.getJSONObject( name = "duration").getInt( name = "value")
            val steps = tramo.getJSONArray( name = "steps")

            // Metros y maniobra inmediatamente siguiente
            val metrosHastaManiobra = steps.getJSONObject( index = 0)
                .getJSONObject( name = "distance").getInt( name = "value")
            val hayManiobraProxima = steps.length() > 1
            val instruccionActual = parsearInstruccion(steps.getJSONObject( index = 0))
                .substringBefore( delimiter = " · ")

            val instruccionSiguiente: String
            if (hayManiobraProxima) {
                instruccionSiguiente = parsearInstruccion(steps.getJSONObject( index = 1))
                    .substringBefore( delimiter = " · ")
            } else {
                instruccionSiguiente = ""
            }
        }
    }
}

```

```

// "Después" muestra lo que viene tras la próxima maniobra
val siguiente: String
if (steps.length() > 2) {
    val texto = parsearInstruccion(steps.getJSONObject(index = 2)).substringBefore(delimiter = " · ")
    siguiente = "Después: $texto"
} else {
    siguiente = "Después: llegarás al destino"
}

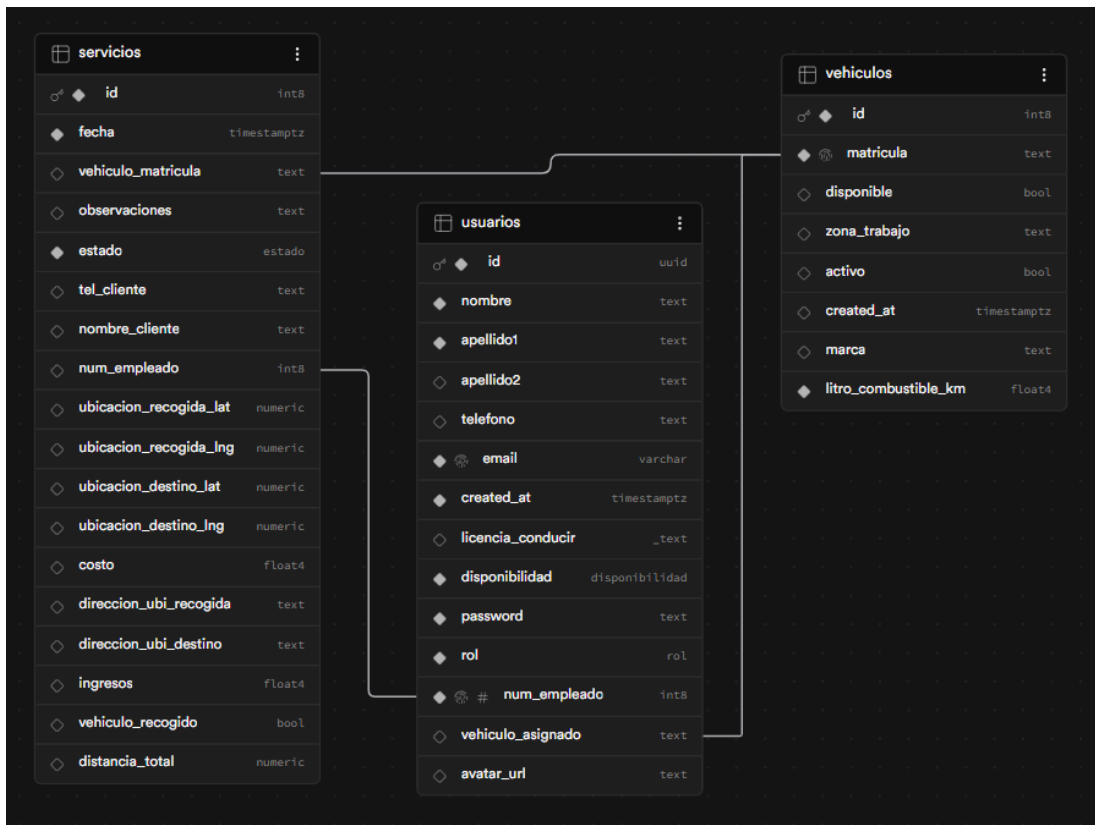
val puntos = decodificarPolyline(
    encoded = obj.getJSONArray(name = "routes").getJSONObject(index = 0)
        .getJSONObject(name = "overview_polyline").getString(name = "points")
)

withContext(context = Dispatchers.Main) {
    dibujarRuta(puntos)
    actualizarUI(
        distancia, tiempo, siguiente, segundos,
        distanciaMetros, hayManiobraProxima,
        metrosHastaManiobra, instruccionActual, instruccionSiguiente
    )
}
} catch (e: Exception) {
    withContext(context = Dispatchers.Main) {
        binding.tvInstruccion.text = "Error: ${e.message}"
    }
}
}
}

```

([MapGPS.kt](#)) Estas dos capturas del código que integra Google Maps en la aplicación móvil, obtiene la ubicación actual del operador y calcula la ruta hasta el punto de recogida del servicio.

### Anexo 3: Relación de tablas en Supabase



Estos códigos definen la estructura principal de la base de datos del sistema.

La tabla de **usuarios** almacena la información de los trabajadores, diferenciando entre administradores, usuarios y operadores mediante el campo rol, también permite ver que vehículo tienen asignado.

La tabla de **vehículos** permite gestionar las grúas disponibles.

La tabla de **servicios** almacena la información de cada servicio en carretera gestionado por la empresa. El campo estado permite controlar la evolución del servicio pasando por diferentes fases.